Build Brief: Three Classics (Connect Four, Checkers, Backgammon)

You're building my submission for the **Chess.com Product Engineering Internship take-home**. It's a small full-stack web app with a home page and three games you play against the computer: **Connect Four, Checkers and Backgammon**. Reviewers will clone the repo, run one command and play. They'll also read the code, the docs and this conversation, so write code and commit messages you'd want a senior engineer to see.

Read this whole brief and every file in `docs/reference/` before writing any code.

---

0. How to work

1. **Plan first.** Start by restating the plan as milestones and listing any open questions. Wait for my approval before writing code.
2. **Build one game at a time, end to end.** Rules, tests, AI, UI and result modal for one game must be done before the next one starts. If time runs short, two polished games beat three broken ones.
3. **Don't stop for small decisions.** Pick the sensible option and log it in `docs/DECISIONS.md`. Ask me only before you:
    - change the stack
    - add a dependency that isn't on the approved list (Section 9)
    - cut a core feature
4. **Prove it works.** Never call a milestone done until its tests pass and you've taken screenshots and looked at them (Section 11).
5. **Commit small and often.** Write commit messages in the imperative mood, and commit at least once per milestone.
6. **Write a project `CLAUDE.md` in Milestone 1.** Keep it to 40 lines or fewer and cover the stack, commands, folder layout, the layout rules from Section 7.3 and the anti-slop list from Section 7.6. Then later sessions keep the conventions.
7. **The rules in Section 5 win over the PDFs.** If the reference rulebooks disagree with this brief, follow this brief. It corrects known problems in those files.

Reference files (`docs/reference/`)

File	What it is	Notes
`ChessCom_Product_Engineering_Internship_Take_Home_Assignment.pdf`	The assignment	Hard requirements. See Section 1.
`checkers-rules.pdf`	One-page American checkers rules	Good source. Section 5.2 adds the edge cases it leaves out.
`backgammon_Official-Rules.pdf`	Rulebook for a 12-game set	**Backgammon is pages 1–4 only.** Ignore the chess, dominoes, poker and sudoku pages. Its single-game scoring rule is nonstandard, so use Section 5.3.
`A5640_en-in_connect-4-game.pdf`	Hasbro kids' playbook	**Has no win rules.** It only shows the 7×6 grid and 21 discs per colour. Use Section 5.1.
`moodboards/jungle.png`, `ocean.png`, `desert.png`, `winter.png`	Pinterest boards	**Mood and palette reference only.** Never copy, trace, hotlink or ship these images. See Section 7.

Online rule sources

Use these to cross-check. Section 5 is still the final word.

Source	Use for	Notes
Boardgammon — Backgammon Rules	**Primary backgammon source.** It's more complete than the PDF: max-dice rule, closeout, bear-off edge cases, doubling cube.	Its bar-entry line ("a 1 enters on the opponent's 24-point") contradicts its own numbering. Use Section 5.3: a die of *n* enters on the **opponent's n-point**, which is the mover's point `25 - n`. Ignore the physical and

Source	Use for	Notes
		tournament procedure (cups, cocked dice, clocks, money-play side rules).
wikiHow — How to Play Connect 4	Connect Four basics	Cross-checked against officialgamerules.org and Wikipedia. Section 5.1 combines them.

---

1. Hard requirements (from the assignment)

- **Stack:**
  - Infrastructure: Docker + Kubernetes
  - Database: PostgreSQL
  - Backend: Java 21+ with Spring Boot
  - Frontend: React + TypeScript (Vite), HTML, CSS
- **One-command local run:** `docker compose up --build`, then open `http://localhost:8080`.
- **Kubernetes manifests** in `k8s/` that run the same stack on a local cluster (kind or Docker Desktop).
- **Submission docs:**
  - `README.md`: how to run, how to test, architecture overview
  - `docs/DECISIONS.md`: main decisions, trade-offs, what I'd do next
  - `docs/TIME_LOG.md`: time per milestone
  - `docs/agent-transcripts/`: I'll export the transcripts there at the end
- **The reviewers' stated value:** "a working, well-built core game over covering every feature." Correctness and feel come first. Extras come last.
- **Time budget:** the assignment expects about 4 hours of my time. Keep the scope disciplined, and present the three games as one shared engine design (Section 4), not three separate apps.

---

2. Priorities, in order

1. **Rules are 100% correct.** Every legal move is allowed and every illegal move is impossible. Wins and draws are detected correctly.

2. **Playable layout.** The board is centered, fully visible without scrolling and easy to read. Cells and pieces never change size. The background never competes with the board.
3. **Clean, restrained design.** Each page gets one theme, a calm palette and simple controls.
4. **Natural motion.** Pieces move smoothly, and each theme has slow ambient animation (leaves, water, sand, snow).
5. **Everything else.**

---

3. Product scope

3.1 Pages and themes

Route	Page	Theme	Ambient motion
`/`	Home	**Jungle**	Fronds sway, mist drifts, light shafts
`/connect-four`	Connect Four	**Ocean**	Light rays shift, bubbles rise, slow surface shimmer
`/checkers`	Checkers	**Winter**	Layered snowfall with gentle wind
`/backgammon`	Backgammon	**Desert**	Wind-blown sand grains, haze drifts over the dunes

Why each theme fits:

- **Connect Four → Ocean.** The classic frame is blue, and discs dropping read like falling through water.
- **Backgammon → Desert.** It's one of the oldest board games, and its warm wood and sandstone palette suits the board.
- **Checkers → Winter.** A crisp two-tone board reads naturally as ice and slate. Snowfall is the calmest motion for a game that needs focus.
- **Home → Jungle.** It's the lush "hub" you step out of into the other three worlds.

3.2 Home page

- Project name as a small, confident wordmark (working name: **"Three Classics"**), plus a one-line subtitle. No marketing copy.
- Three tall portrait cards, like windows into each game's world. Each card shows:
  - a static thumbnail of that game's theme scene, rendered by the same scene component in thumbnail mode
  - the game name and a one-line description
  - a **Play** button, or **Continue** if an unfinished game exists
  - a small record line, e.g. `W 4 · L 2 · D 1`
- Hovering a card starts its scene's ambient motion and lifts it slightly. Use transform only, so layout never shifts.
- Keyboard: Tab moves between cards, Enter opens a game.

3.3 Features every game must have

- **Mode:** vs Computer (default) or 2 Players on one device. Two-player mode is nearly free with a server-side engine and makes manual testing easy.
- **Difficulty:** Easy / Medium / Hard, chosen before the game and shown during it. Changing it mid-game asks "Restart on Hard?"
- **Side or order:**
  - Connect Four: Go first / Go second
  - Checkers: Play Dark (moves first) / Play Light. The board flips so the human is always at the bottom.
  - Backgammon: the opening roll decides who starts
- **Turn indicator** and a **"Computer is thinking…"** state. Show the AI move after a minimum delay of about 500 ms so it feels natural, even if it was computed instantly.
- **Legal-move highlighting:** select a piece to see where it can go. Clicking an illegal target does nothing (a subtle shake is fine). It never errors.
- **Last-move highlight**, so the player can see what the computer just did.
- **Restart** button. If a game is in progress, confirm first: "Restart this game?" → Restart / Cancel.
- **Home** button.
- **Rules** button that opens a side drawer with a short how-to-play for that game. Write it in your own words from Section 5. Don't paste rulebook text.
- **Result modal** when the game ends:
  - Title: **You win** / **Computer wins** / **Draw**. In 2-player mode: **Dark wins**, **Player 1 wins**, etc.
  - Subtitle with the reason, e.g. "Four in a row (diagonal)", "No legal moves left", "Gammon (2 points)", "Draw by 40-move rule"
  - Buttons: **Play again** (primary, same settings), an inline **Easy / Medium / Hard** switch so you can play again at a new level, and **Home**

    - Enter = Play again. Esc closes the modal so you can study the final board, and a "Show result" pill reopens it.
    - First let the player see the ending: highlight the winning line or final move for about 800 ms, then open the modal.
- **Resume after refresh:** store the active game id per game type in `localStorage`, wrapped in try/catch. On load, fetch that game. If it's missing or finished, offer a new game.
- **Settings** (small gear menu):
    - Ambient motion on/off. Default is on, but off when `prefers-reduced-motion` is set.
    - Sound on/off, if sound is built (stretch goal)

---

4. Architecture

4.1 Principle: the server is the referee

All rules and all AI live in the Java backend. The frontend **never** decides legality:

- It renders the state and the server-provided `legalMoves`.
- It sends the chosen move back.
- The server validates and applies the move, runs the AI and returns the result.

This keeps one source of truth, makes the rules unit-testable, and mirrors how a real game site prevents cheating. Note it in DECISIONS.md.

Multi-step moves. For checkers multi-jumps and backgammon turns, the server returns each legal move as a complete **path or sequence of steps**. The frontend lets the user build a move click by click, accepting only steps that follow a legal prefix, then submits the completed move. The rules logic stays in one place, and the UI is still instant and supports in-turn undo.

4.2 Backend layout (Java 21+, Spring Boot)

```
None
backend/src/main/java/.../
  api/            REST controllers, request/response DTOs, error handler
(ProblemDetail)
  core/           Shared abstractions:
                    GameEngine<S, M>  initialState, legalMoves, apply, outcome
                    AiPlayer<S, M>    chooseMove(state, difficulty, rng,
timeBudget)
                    Outcome, Difficulty, Side, IllegalMoveException
```

```
  connectfour/    Engine + AI (bitboard search is fine inside the AI)
  checkers/       Engine + AI
  backgammon/     Engine + AI + dice
  service/        GameService: load → validate → apply → AI reply → persist (one
transaction)
  persistence/    JPA entities, repositories
resources/db/migration/   Flyway migrations
backend/src/test/java/... Rules tests per game, AI tests, API tests
```

- **Engines are pure.** Use immutable state (records) and `apply()` returns a new state. Inject `Random` (seeded in tests). Never use `Math.random()` or `new Random()` inside engine logic.
- **Dice are rolled on the server** with `SecureRandom` in production and seeded `Random` in tests.
- **Optimistic locking:** each game has a `version`. A move request carries the version it was based on, and the server returns 409 on mismatch so the client can refetch. This stops double-clicks from applying twice.
- **AI time budget:** Hard must answer in about 1 s and never more than 2 s. Use iterative deepening with a deadline.

4.3 API (JSON)

```
None
POST /api/games                 { type, mode, difficulty, humanSide }  → GameView
GET  /api/games/{id}                                                   → GameView
POST /api/games/{id}/moves      { version, move }                      → { game:
GameView, events: Event[] }
GET  /api/stats                                                        → per game
type × difficulty: wins, losses, draws
GET  /actuator/health
```

- **Request fields:**
  - `type`: `CONNECT_FOUR | CHECKERS | BACKGAMMON`
  - `mode`: `VS_AI | LOCAL`
  - `difficulty`: `EASY | MEDIUM | HARD`
- **`GameView`** contains: id, type, mode, difficulty, version, status, sideToMove, a per-game `state` object, `legalMoves`, `outcome`, `lastMove`.
- **`events`** is the ordered list of what happened in this request, e.g. human move → AI roll → AI steps. The frontend animates them in order, then snaps to the final `game` state, which is the source of truth.
- **Backgammon rolls** are a move: `{ kind: "ROLL" }`. Keep one move endpoint.

- **Dev/test profile only:** `POST /api/dev/games` accepts a custom position, so tests can start near a win and check the modal. It must not be exposed in the default profile.

4.4 Database (PostgreSQL + Flyway)

- `games`: id (uuid), type, mode, difficulty, human_side, status, winner, result_reason, points, state (jsonb), version, created_at, updated_at
- `moves`: id, game_id, ply, actor (HUMAN/AI/P1/P2), move (jsonb), created_at
- Stats are computed from finished `games`. There are no accounts; this is a single local player.

4.5 Frontend layout (React + TypeScript + Vite)

```
None
frontend/src/
  api/            Typed client + DTO types
  app/            Router, layout shell, settings context
  components/     Shared UI only: Button, SegmentedControl, Modal, Drawer, TopBar,
ConfirmDialog, Toast
  games/
    connect-four/ Board (SVG), hooks, game page
    checkers/
    backgammon/
  themes/
    tokens.css    Spacing, type, motion tokens
    jungle/ ocean/ desert/ winter/   Palette vars + Scene component (SVG layers +
one canvas)
  pages/Home.tsx
```

Use plain CSS with CSS variables and CSS Modules. No component kits.

---

5. Rules (authoritative)

5.1 Connect Four

1. **Board:** a vertical grid, 7 columns × 6 rows. Each player has 21 discs.
2. **Turns:** players alternate. Each turn, drop one disc into any column that isn't full. It falls to the lowest empty slot.
    - The player picks a column, never a cell. A full column can't be chosen.
    - A disc never moves or comes out once dropped.
3. **Win:** the first player with **four in a row** wins immediately. The line can be horizontal, vertical, or either diagonal.

    - A line of **five or more counts as a win**.
    - If one drop completes two lines at once, highlight every winning disc.
4. **Draw:** all 42 slots are filled with no four in a row.
5. **Who goes first:** the rules don't fix a colour to start.
    - In the first game, the human chooses Go first or Go second. The first player always uses coral.
    - **Play again alternates the starting player** by default, as the rules suggest for fairness. The pre-game choice can override this.
6. **Board orientation:** column 0 is on the left.
7. **Solved-game note, for the AI and DECISIONS.md:** with perfect play the first player wins by opening in the center column. Columns next to the center draw, and outer columns lose.
    - Hard should therefore open in the center when it moves first.
    - Easy and Medium should vary their openings.

5.2 Checkers (American checkers / English draughts)

1. **Board:** 8×8. Play only on the 32 dark squares. Each player's **bottom-left corner square is a dark, playable square**.
2. **Setup:** 12 pieces each on the dark squares of the three rows nearest each player. The two middle rows start empty. **Dark moves first.**
3. **Men** move one square diagonally **forward** onto an empty dark square.
4. **Kings** move one square diagonally forward **or backward**.
    - **No flying kings.** This is not international draughts.
5. **Captures:**
    - **Jumping:** a piece jumps diagonally over an adjacent opponent piece into the empty square right beyond it, and the jumped piece is removed. Men capture forward only; kings capture both directions.
    - **Mandatory:** captures are **mandatory**. If any capture is available, only capture moves are legal.
    - **Free choice:** if several captures are available, the player **may choose any of them**. There is no requirement to take the longest sequence or the most pieces.
    - **Multi-jumps:** once a piece starts jumping, it **must keep jumping** with that same piece while further jumps exist. It may not stop partway, and it can't jump the same piece twice.
6. **Crowning:**
    - A man reaching the far row becomes a king. Show it as a stacked, double-height piece with a small crown mark, so it isn't colour-only.
    - **If a man is crowned during a jump sequence, the move ends there.** It can't continue jumping as a king that turn.
7. **Loss:** a player loses if they have **no pieces** or **no legal move** on their turn.
8. **Draws** (needed for vs-computer play):

- **40-move rule:** 40 moves by each side (80 plies) with no capture and no man move (only kings moving).
- **Threefold repetition:** the same position with the same side to move appears three times.

5.3 Backgammon

Numbering: points are numbered 1–24 from the side to move's perspective. Their home board is points 1–6 and they move from 24 toward 1. On screen, the human's home board is **bottom-right**. Human checkers travel counter-clockwise: from the top-right, across the top, down the left, and along the bottom into home. The bar runs down the middle, and the bear-off tray is on the right.

1. **Setup** (each player, own perspective): 2 checkers on the 24-point, 5 on 13, 3 on 8, 5 on 6. That's 15 each.
2. **Opening roll:**
    - Each side rolls one die, and ties re-roll.
    - The higher die moves first, **using both of those dice as its first roll**.
    - After that, players alternate rolling two dice.
3. **Moving:**
    - **Separate moves:** each die is a separate move. One checker can use both dice (as long as the intermediate landing point is open), or two checkers can use one die each.
    - **Doubles:** play the number **four times**.
4. **Landing:**
    - **Allowed:** a checker may land on an empty point, a point it already holds, or a point with exactly **one** opposing checker (a blot).
    - **Hitting:** landing on a blot **hits** it, and the blot goes to the bar.
    - **Blocked:** a point with 2+ opposing checkers can't be landed on, but it can be jumped over.
5. **Using the roll** (where most implementations go wrong):
    - **Maximum dice:** the player **must use as many dice as legally possible**. That means both dice, or up to four for doubles.
    - **Larger die:** if only one die can be used and either could be, **the larger die must be used**.
    - **No legal moves:** the turn is forfeited. Show a short toast like "No legal moves — turn passes."
    - **Implementation:** generate every complete step sequence by DFS over both die orders. Keep only the sequences that use the maximum number of dice. If that maximum is 1 and both dice are individually playable, keep only the larger-die moves. Deduplicate by resulting position.
6. **The bar:**

    - A player with checkers on the bar **must enter all of them before moving any other checker**.
    - A checker enters on point `25 - die`, i.e. in the opponent's home board, which is the mover's points 19–24.
    - If the entry points are blocked for every usable die, the turn is over.
    - **Closeout:** if all six points of the opponent's home board are made, the player can't enter at all. They keep passing, without needing to roll, until a point opens. Show "Closed out — turn passes."
7. **Bearing off:**
    - **When:** only when **all 15** of the player's checkers are in their home board, with none on the bar.
    - **Exact die:** a die of `d` bears off a checker from point `d`.
    - **No checker on point `d`:** if a checker sits on a higher point, the player must use the die to move a checker from a higher point. If no checker is on any higher point, they must bear off from the **highest occupied** point.
    - **Moving instead:** the player may move a checker inside the home board instead of bearing off, as long as the maximum-dice rule is still met.
    - **Hit while bearing off:** the checker must re-enter and travel back home before bearing off resumes.
8. **Winning:** the first player to bear off all 15 wins. Use **standard scoring** and show it in the modal:
    - **Single game, 1 point:** the loser has borne off at least one checker.
    - **Gammon, 2 points:** the loser has borne off no checkers.
    - **Backgammon, 3 points:** the loser has borne off none **and** still has a checker on the bar or in the winner's home board.
    - The PDF's alternative "count the loser's remaining checkers" rule is **not** used. Log this in DECISIONS.md.
9. **Doubling cube: not in v1.** It's a stretch goal (Section 10). Log it in DECISIONS.md as a deliberate cut.
    - We play single games, not matches, so ignore the Crawford rule.
    - Also ignore the money-play side rules: Jacoby, beavers, raccoons and automatic doubles.
10. **Ending a turn:** in physical play, a move stands once you pick up your dice. Here, the **Confirm** button is that moment. Before Confirm, Undo is free. After Confirm, the turn is final.

Backgammon UI specifics:

- **Rolling:** a **Roll** button for the human. Dice tumble in 3D for about 600 ms. Show both dice, and dim each one as it's used. Doubles show four pips or a ×2 badge.
- **Moving checkers:**
    - Click a checker (or the bar) to see its destinations. Click a destination to move.

    - Show **Undo** (within the turn) and **Confirm**. Confirm turns on only when a complete legal turn has been built.
    - Optional: double-click a checker to play the larger usable die automatically.
- **Counts and stacks:**
    - Show each side's **pip count**.
    - If a stack holds more than 5 checkers, overlap them or show a count badge on the 5th. The stack must never overflow its point.
- **Bear-off tray** shows the count, e.g. `7 / 15`.

---

6. Computer opponent

Shared rules for every AI:

- It only ever plays moves taken from the engine's `legalMoves`.
- Its randomness comes from the injected `Random`.
- Hard respects the time budget.

Difficulty targets: **Easy**, a beginner wins most games. **Medium**, a casual player wins sometimes. **Hard**, plays solid and rarely blunders.

	Easy	**Medium**	**Hard**
Connect Four	Mostly random. Takes an immediate win or block only ~50% of the time.	Minimax depth 4–5 with a window-scoring heuristic and a center preference	Negamax with alpha-beta, bitboards, center-first move ordering, transposition table, iterative deepening (~10–12 ply in budget)
Checkers	Random legal move (forced captures still apply)	Minimax depth 4. Eval: material (man 100, king ~160) + advancement	Alpha-beta + iterative deepening (~8–10 ply). Eval: material, advancement, back-row guard, center control, mobility, endgame king activity

	Easy	Medium	Hard
Backgammon	Random legal turn	Greedy one-ply: best resulting position by heuristic (pip count, blots exposed to direct shots, points made, primes, home-board strength, checkers on bar)	Heuristic eval + 2-ply expectimax over all 21 opponent rolls, with race vs contact phase detection

If the moves are close in value, Medium may pick randomly among near-equal moves so games vary.

---

7. Visual design

7.1 Direction

The goal is **quiet and atmospheric, not flashy**. Think of a calm nature documentary title card, not a gaming landing page.

- **The board is the hero.** The scene is a backdrop.
- **Scenes are built in code, not photos:**
  - layered SVG silhouettes with atmospheric perspective (farther layers are lighter, hazier and less saturated)
  - CSS gradients
  - one lightweight `<canvas>` per page for particles
- **Photo slot:** structure each scene so a licensed photo could later be dropped in as the bottom layer, under the same scrim, via one CSS variable.

7.2 Theme palettes

These are pulled from the moodboards. Treat them as starting points and check contrast.

Theme	Background range	Surface / board	Accents
Jungle (home)	mist `#D7E4D9` → canopy `#4C7A55` → deep `#0F2619`	card surface `#F4F1E8` at 92%	bird-of-paradise orange `#E8742C` (sparingly), lagoon teal `#2F8F86`

Theme	Background range	Surface / board	Accents
Ocean (Connect Four)	surface `#5CC8D0` → teal `#138A9E` → deep `#0B4F63` → abyss `#04202F`	frame `#0D3B4F`, empty holes `#062A38` with inner shadow	discs: coral `#FF7A5C`, sand-gold `#F2C14E`, each with an embossed inner ring
Desert (Backgammon)	dusk lavender `#B9A3C9` → peach `#F3C49A` → dunes `#E9B98A` / `#D98A52` / `#B5562A`	frame walnut `#5A2E1B`, field sand `#E8CFA4`, points alternating terracotta `#C4562A` / umber `#8A5A3B`	checkers: ivory `#F6EFE2`, obsidian `#2B1A12`; dice ivory with dark pips
Winter (Checkers)	blue hour `#1F2F4D` → `#5F7FA8` → horizon `#C9D8EA` → snow `#EEF3F8`	light squares `#E6EEF6`, dark squares `#3B5573`, frame `#22324A`	pieces: Dark = charcoal-navy `#151D2B` with frosted rim, Light = snow `#FBFDFF` with ice edge `#A9C6E2`; warm lamp accent `#F2C48D` for highlights

Typography:

- **Fraunces** for display text (page titles and modal titles only). **IBM Plex Sans** for all UI text.
- Self-host both via `@fontsource`, since the app must work offline in Docker.
- Use tabular numbers for counts.

7.3 Layout rules for boards (non-negotiable)

1. **Each board is a single `<svg>` with a fixed `viewBox`.**
   - Cells, points and pieces are drawn in that fixed coordinate system, so their size can never depend on content.
   - The SVG is scaled as a whole by its container.
   - Pieces render at a fixed fraction of the cell (e.g. radius = 40% of cell size), whether or not neighbours exist.
2. **Board sizing:**
   - The board container has a fixed `aspect-ratio`: 1/1 for checkers, 7/6 plus frame for Connect Four, about 4/3 for backgammon.

    - Size it to the largest box that fits the viewport minus the top bar, side panel and gutters, capped at about 760 px tall.
    - It is **horizontally and vertically centered** in its column.
3. **Nothing reflows during play.** Selection, hover and highlight states use SVG rings, fills, transforms or opacity, never borders or padding that change geometry.
    - Status text, the dice area and the "thinking…" line **reserve their height** up front, so messages appearing never push the board.
    - The board only resizes when the window resizes.
4. **Screen structure:**
    - A top bar (about 56 px): Home on the left, game name in the center, difficulty, Restart, Rules and Settings on the right.
    - The board, centered.
    - On screens ≥ 1100 px wide, a 280 px side panel: turn, status, captured pieces or pip counts, and move list. Below that width, the panel moves under the board.
5. **The background stays a backdrop.** Behind the board area, apply a soft radial scrim that darkens and desaturates the scene by about 40–60%.
    - The board sits on a solid surface with **one** soft shadow.
    - All text meets WCAG AA contrast.
6. **Minimum targets:** the board is fully visible with no scrolling at **1280×720**, and still playable at **390×844** (phone portrait).

7.4 Ambient scenes

Jungle (home), back to front:

1. A misty gradient sky with far karst ridges, hazy and low-contrast.
2. One or two mist bands drifting horizontally on 60–90 s loops.
3. Soft light shafts whose opacity breathes over 8–12 s.
4. Foreground fronds (monstera, palm, bamboo) framing the edges. They sway ±1.5–3° from the stem (`transform-origin` at the base) over 6–10 s, staggered.
5. A rare falling leaf: at most 3 on screen at once, one every 6–10 s.

Ocean (Connect Four):

1. A depth gradient.
2. 4–6 soft god rays from the surface, swaying slowly over 12–20 s.
3. A faint surface shimmer at the top.
4. 15–30 small bubbles rising with a slight wobble.
5. Reef silhouettes at the bottom edges.
6. Optional: a faint manta silhouette gliding across about once a minute.

Desert (Backgammon):

1. A dusk sky with a low, pale sun.
2. Three dune layers with atmospheric perspective.
3. Fine sand grains blowing horizontally, sparse, with an occasional 2 s gust.
4. Very slow haze drift.

Winter (Checkers):

1. A blue-hour sky.
2. Hazy snow-capped ridges and snow-laden pine silhouettes.
3. Snowfall in **three depth layers** (far: tiny and slow; near: larger, faster and slightly soft), with sinusoidal drift and slowly changing wind. About 120–200 flakes total, scaled down on small screens.
4. Optional: a barely-there aurora band at 10–15% opacity.

Performance and accessibility:

- One `requestAnimationFrame` loop per page, aware of devicePixelRatio.
- Pause when the tab is hidden, and cut particle counts on small screens.
- With `prefers-reduced-motion` or the setting turned off, show a static scene.
- Target 60 fps on a normal laptop.

7.5 Piece motion

- **Motion tokens:**
  - `--dur-fast: 150ms` (hovers)
  - `--dur-base: 250ms` (UI)
  - `--dur-piece: 300–400ms` (moves)
  - easing `cubic-bezier(.2,.8,.2,1)`
- **Connect Four:**
  - A preview disc hovers above the column under the cursor.
  - Dropped discs fall with gravity (ease-in) and land with one small settle bounce.
  - The winning four get a slow pulse ring.
- **Checkers:**
  - The selected piece lifts (scale 1.06 and a deeper shadow, transform only).
  - Moves slide diagonally. Multi-jumps animate hop by hop.
  - Captured pieces shrink and fade.
  - Crowning stacks the second disc with a tiny frost sparkle.
- **Backgammon:**
  - Dice tumble.
  - Checkers slide to their point. A hit checker slides to the bar. Bearing off slides into the tray.
  - Legal destination points glow softly.
- **Result modal:**

    - It fades and scales from 0.98 to 1 over 200 ms while the backdrop dims.
    - One understated themed flourish: a brief snow flurry, a rise of bubbles, or a sand gust. Nothing more.
- With reduced motion on, piece moves become quick crossfades.

7.6 Anti-slop rules

Don't use:

- purple or indigo "AI gradients"
- gradient text
- neon glows
- glassmorphism everywhere
- emoji as icons
- sparkle icons
- confetti cannons
- rounded-everything with heavy drop shadows
- bouncing UI chrome
- marketing hero sections
- fake stats or placeholder or lorem text
- "Powered by AI" badges
- more than one font family besides the display face
- more than two accent colours per theme

Do:

- generous whitespace on an 8 px spacing scale
- short plain copy ("New game", "Your turn", "You win")
- one icon set (Lucide) at one stroke weight
- visible focus rings
- colour plus shape for state, never colour alone
- consistent 8–12 px radii

No Chess.com or Hasbro logos or branding. The game is called "Connect Four" in plain text only.

---

8. Accessibility and input

- Every interactive cell or point is a focusable element with an `aria-label`, e.g. "Column 4, 3 discs" or "Point 13, 5 light checkers".
- **Connect Four:** ←/→ choose a column and Enter drops a disc.

- **Checkers and Backgammon:** Tab or arrow keys move focus, and Enter selects or moves. Mouse-first is fine, but keyboard must work.
- The modal traps focus and returns it to the board when closed.

---

9. Dependencies

Approved dependencies, by area. Ask before adding anything else.

- **Backend:**
  - `spring-boot-starter-web`, `-data-jpa`, `-validation`, `-actuator`
  - `flyway-core` (+ postgres module), `postgresql`
  - `spring-boot-starter-test`, Testcontainers (Postgres) for integration tests
  - Maven or Gradle **wrapper** (pick one), so no local install is needed
- **Frontend:**
  - `react`, `react-dom`, `react-router-dom`, `typescript`, `vite`
  - `lucide-react`, `@fontsource/fraunces`, `@fontsource/ibm-plex-sans`
  - `vitest`, `@testing-library/react`, `@playwright/test`
  - Optional: `motion` (Framer Motion), only if it clearly simplifies piece animation
- **Not allowed:**
  - Tailwind or other CSS frameworks
  - component kits (MUI, Chakra, shadcn)
  - state libraries (Redux etc.)
  - game or chess engines from npm or Maven. All rules and AI must be our own code.

---

10. Out of scope / stretch goals

Do stretch goals only after everything else is done, and in this order:

1. **Hint button:** ask the Hard AI for a suggested move and highlight it.
2. **Takeback:** undo your last move plus the computer's reply. The move history makes this easy.
3. **Subtle sound effects:** disc clack, dice roll, piece slide. Off by default.
4. **Backgammon doubling cube**, following Boardgammon's basic rules:
   - **Offering:** either player may offer the first double, but only **before rolling** on their own turn.
   - **Take:** stakes double and the taker owns the cube. Only the owner may redouble later.
   - **Pass:** the game ends and the player who passed loses the current cube value.

- **Limits:** the cube is capped at 64. No beavers, automatic doubles or Jacoby rule.
- **Scoring:** gammons and backgammons multiply the cube value.
- **AI:** it offers, takes and passes using eval thresholds.
- **Display:** show the cube centered (value 1) or on its owner's side.

Out of scope: accounts, login, online multiplayer, WebSockets, chess, leaderboards.

---

11. Testing and verification

11.1 Backend (JUnit), required per game

Connect Four:

- wins horizontally, vertically and on both diagonals, including at the edges
- a line of five counts as a win
- a double line is fully highlighted
- Play again alternates the starting player
- a full column is rejected
- a full board is a draw
- the game stops accepting moves after a win

Checkers:

- setup has 12 per side on dark squares, and Dark moves first
- men can't move backward
- kings move both ways and don't fly
- forced capture removes all non-captures from `legalMoves`
- multi-jump must continue
- the player may pick a shorter capture over a longer one
- **a man crowned mid-jump ends the move**
- no legal moves loses
- the 40-move draw and threefold repetition work

Backgammon:

- setup has 15 per side on 24/13/8/6, and each starting pip count is **167**
- bar entry for a die of *n* lands on the opponent's n-point
- a closeout passes the turn
- the opening roll uses both opening dice
- 2+ checkers block a point

- hitting sends a blot to the bar
- the bar must be cleared first
- blocked entry forfeits the turn
- doubles give four moves
- **the max-dice rule**
- **the larger-die rule when only one die can be played**
- no bear-off unless all 15 are home
- bearing off with a higher die from the highest point
- having to move from a higher point instead of bearing off
- a hit during bear-off stops bearing off
- single, gammon and backgammon scoring

Invariants (soak test):

- 500 AI-vs-AI games per type (Easy vs Easy, seeded) run to completion with no exceptions.
- Checkers piece counts never increase.
- Backgammon always has 15 per side across board + bar + off.
- Every AI move is in `legalMoves`.

AI:

- Hard Connect Four always takes an immediate win, and always blocks when the opponent has exactly one immediate winning threat.
- Hard responds within the 2 s cap on sample positions.

API:

- an illegal move → 400 with a clear ProblemDetail
- a stale version → 409
- the dev endpoint isn't exposed in the default profile

11.2 Frontend

Vitest:

- the legal-prefix matcher for multi-step moves
- the board coordinate mapping, including the checkers flip and backgammon orientation

Playwright e2e:

- Home → each game → make a move → the AI responds → Restart → Home

- Start from a near-win dev position, win, see the modal, click Play again, and confirm a fresh board

Playwright layout test for each game at **1280×720, 1440×900, 1920×1080 and 390×844**:

- the board is fully inside the viewport, with no scrollbars
- the board is centered in its column within 2 px
- **all cells or points have identical size (±0.5 px)**
- **cell rects are unchanged before and after a move and after the AI reply**
- nothing overlaps the board

11.3 Visual review (you must do this)

After each game milestone and at the end:

- Take Playwright screenshots of every page at the four viewports, plus the result modal.
- **Open the screenshots and look at them.**
- Fix anything misaligned, cramped, low-contrast or busier than this brief allows before moving on.
- Save the final set to `docs/screenshots/` and use a few in the README.

---

12. Infrastructure

Dockerfiles:

- **Backend:** multi-stage. Build with the wrapper, then run on `eclipse-temurin` JRE as a non-root user.
- **Frontend:** multi-stage. Node builds the app, then `nginx:alpine` serves it and proxies `/api` to the backend.

`docker-compose.yml`:

- three services: `db` (postgres:16-alpine with a named volume), `backend` and `frontend`
- healthchecks on all three
- `depends_on: condition: service_healthy`
- the app at `http://localhost:8080`
- defaults that work with zero config, with credentials in `.env.example`

`k8s/` (Kustomize):

- namespace

- Postgres StatefulSet + PVC + Secret
- backend Deployment + Service, with readiness/liveness probes on `/actuator/health`
- frontend Deployment + Service
- ConfigMap
- a `kustomization.yaml`

The README shows the kind flow: create the cluster, build the images, `kind load`, `kubectl apply -k k8s/`, then port-forward. A `make k8s-up` / `make k8s-down` helper is welcome.

Makefile: `make up`, `make down`, `make test`, `make e2e`, `make k8s-up`.

---

13. Docs to write

`README.md`:

- a one-paragraph pitch and a screenshot strip
- **Quick start:** **`docker compose up --build`**
- running the tests
- Kubernetes
- an architecture section: a Mermaid diagram of browser → nginx → Spring Boot → Postgres, plus the "server is the referee" explanation
- the project layout
- how the AI works, briefly per game

`docs/DECISIONS.md`:

- the key decisions and why:
  - server-authoritative engine
  - legal-move paths with prefix matching
  - SVG boards
  - procedural scenes instead of photos
  - three games via a shared `GameEngine` interface
- trade-offs, including no doubling cube, standard scoring versus the PDF, and no accounts
- what I'd do next with more time

`docs/TIME_LOG.md`: a table of milestones with start and end times taken from commit timestamps. I'll correct it.

---

14. Milestones

1. **Skeleton.**
   - Repo structure, `CLAUDE.md`, Docker Compose with all three services healthy, a `/api/games` stub persisting to Postgres, and the React shell with routing and the theme token system.
   - **Gate:** `docker compose up --build` works from a clean clone.
2. **Connect Four, complete.**
   - Engine, tests, AI ×3, ocean scene, board, all shared features from Section 3.3, and the result modal.
   - This sets the pattern for the other two games.
   - **Gate:** all Section 11 tests for Connect Four pass and the screenshots have been reviewed.
3. **Checkers, complete.** Same gate.
4. **Backgammon, complete.** Same gate. This is the hardest rules engine, so get the rule tests green before polishing the UI.
5. **Home page.** Jungle scene, game cards with thumbnails, stats and Continue.
6. **Polish pass.**
   - Motion review, reduced-motion check, keyboard pass, contrast check.
   - A full screenshot review, then fixes.
7. **Kubernetes + docs.** Manifests tested on kind, plus the README, DECISIONS and TIME_LOG.
8. **Stretch goals** (Section 10), only if time remains.

Definition of done (whole project)

- [ ] `docker compose up --build` on a clean machine → all three games playable at `localhost:8080`
- [ ] All backend tests, Vitest and Playwright (e2e + layout) pass
- [ ] Every game: correct rules, three difficulty levels, result modal with Play again / Home, Restart, Rules drawer, resume after refresh
- [ ] Boards centered, fully visible at 1280×720, cells and pieces fixed-size, verified by the layout test
- [ ] Ambient scenes on all four pages, paused on hidden tabs, off with reduced motion
- [ ] `kubectl apply -k k8s/` works on kind
- [ ] README, DECISIONS and TIME_LOG are written. Remind me to export the agent transcripts into `docs/agent-transcripts/`.